

SV9 - Digital Verification using SV & UVM

=====Assignment (6)=====

Applying UVM for a full environment

Name
Ibrahim Mohamed Mohamed Hanafy

ALSU environment

-seq Item

```
class ALSU_seq_item extends uvm_sequence_item;
  `uvm_object_utils(ALSU_seq_item)
```

```
rand logic rst, red_op_A, red_op_B, bypass_A, bypass_B, cin, direction, serial_in;
rand opcode_e opcode;
rand logic signed [2:0] A, B;
```

```
logic [15:0] leds;
logic signed [5:0] out;
```

```
rand opcode_e my_opcodes[6];
```

```
//Constraints=====
constraint Reset { //off for 90% of the time
  rst dist {0:/90 , 1:/10};
}
```

```
constraint Input { //MAXs and Zero 80% of the time for
  if((opcode == ADD) || (opcode == MULT)){
    A dist {MAXPOS:=40,
      MAXNEG:=40,
      ZERO:=40,
      [-3:-1], [1:2] := 15};
    B dist {MAXPOS:=40,
      MAXNEG:=40,
      ZERO:=40,
      [-3:-1], [1:2] := 15}; //15/135
  }
}
```

```
constraint Bypass { //off for 90% of the time
  bypass_A dist {0:/90 , 1:/10};
  bypass_B dist {0:/90 , 1:/10};
}
```

```
constraint Validity { //Invalid for 10% of the time
  opcode dist {INVALID_6:/10,INVALID_7:/10,
    [OR:ROTATE]/80};
  if(opcode inside {OR, XOR}){
    red_op_A dist{0:/50 , 1:/50}; //both red high
    if(red_op_A){
      red_op_B dist{0:/80 , 1:/20};
    }
  }
}
```

```
function string convert2string();
```

```
  return $sformatf("%s\n rst = %b | red_op_A = %b | red_op_B = %b | bypass_A = %b | bypass_B = %b \n c:
  super.convert2string() ,rst, red_op_A, red_op_B, bypass_A, bypass_B, cin, direction, serial_in, opcode);
```

```
endfunction
```

```
function string convert2string_stimulus();
```

```
  return $sformatf("%s\n rst = %b | red_op_A = %b | red_op_B = %b | bypass_A = %b | bypass_B = %b \n c:
  super.convert2string() ,rst, red_op_A, red_op_B, bypass_A, bypass_B, cin, direction, serial_in, opcode);
```

```
endfunction
```

```
constraint Reduction_Operations { //selected has one bit
  if(opcode inside {OR, XOR}){ //unslected has all low
    if(red_op_A){
      A dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
      B == 3'b000;
    }
    else if(red_op_B){
      B dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
      A == 3'b000;
    }
  }
}
```

```
constraint opcode_rand_array {/"constraint 8"
  unique {my_opcodes};
  foreach(my_opcodes[i]){
    my_opcodes[i] inside {[OR:ROTATE]};
  }
}
```

-sequencer

```
class ALSU_sqr extends uvm_sequencer #(ALSU_seq_item);
  `uvm_component_utils(ALSU_sqr)

  function new(string name = "ALSU_sequencer" , uvm_component parent = null);
    super.new(name, parent);
  endfunction

endclass
```

-sequences

```
class ALSU_reset_seq extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_reset_seq)

  ALSU_seq_item seq_item;

  function new(string name = "ALSU_reset_seq");
    super.new(name);
  endfunction

  task body;
    seq_item = ALSU_seq_item::type_id::create("seq_item");

    start_item(seq_item);
    seq_item.rst = 1;      seq_item.cin = 0;
    seq_item.red_op_A = 0; seq_item.red_op_B = 0;
    seq_item.bypass_A = 0; seq_item.bypass_B = 0;
    seq_item.direction = 0; seq_item.serial_in = 0;
    seq_item.opcode = opcode_e'(0);
    seq_item.A = 0; seq_item.B = 0;
    finish_item(seq_item);
  endtask

endclass
```

```

class ALSU_Constrained_main_seq extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_Constrained_main_seq)

  ALSU_seq_item seq_item;

  function new(string name = "ALSU_main_seq");
    super.new(name);
  endfunction

  task body;
    seq_item = ALSU_seq_item::type_id::create("seq_item");

    seq_item.opcode_rand_array.constraint_mode(0); //disable only valid
    repeat(2000) begin
      start_item(seq_item);
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask

```

```

class ALSU_Heavy_opcode_verification_seq extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_Heavy_opcode_verification_seq)

  ALSU_seq_item base_item, seq_item;

  function new(string name = "ALSU_Heavy_opcode_verification_seq");
    super.new(name);
  endfunction

  task body;

    repeat(1000) begin

      base_item = ALSU_seq_item::type_id::create("base_item");
      base_item.constraint_mode(0);
      base_item.opcode_rand_array.constraint_mode(1);
      assert(base_item.randomize());

      foreach(base_item.my_opcodes[i]) begin
        seq_item = ALSU_seq_item::type_id::create($sformatf("seq_item_%0d", i));
        start_item(seq_item); //ready and waiting
        seq_item.A = base_item.A;
        seq_item.B = base_item.B;
        seq_item.cin = base_item.cin;
        seq_item.rst = base_item.rst;
        seq_item.red_op_A = base_item.red_op_A;
        seq_item.red_op_B = base_item.red_op_B;
        seq_item.bypass_A = base_item.bypass_A;
        seq_item.bypass_B = base_item.bypass_B;
        seq_item.direction = base_item.direction;
        seq_item.serial_in = base_item.serial_in;
        seq_item.opcode = base_item.my_opcodes[i];
        finish_item(seq_item); //ship for driver
      end
    end
  endtask

```

-driver

```
class ALSU_driver extends uvm_driver #(ALSU_seq_item);
  `uvm_component_utils(ALSU_driver)

  ALSU_config ALSU_cfg;
  virtual ALSU_if ALSU_vif;
  virtual ALSU_if ALSU_if_golden;
  ALSU_seq_item seq_item_stimulus;

  function new(string name = "ALSU_driver", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    //ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg"); we handle the obje
    if (!uvm_config_db #(ALSU_config)::get(this, "", "CFG", ALSU_cfg)) begin
      `uvm_fatal("build_phase", "Unable to get configuration object")
    end
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    ALSU_vif = ALSU_cfg.ALSU_vif;
    ALSU_if_golden = ALSU_cfg.ALSU_if_golden;
  endfunction
```

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);
  seq_item_stimulus = ALSU_seq_item::type_id::create("seq_item");

  forever begin
    seq_item_port.get_next_item(seq_item_stimulus); // I want, wa

    ALSU_vif.rst = seq_item_stimulus.rst;
    ALSU_vif.cin = seq_item_stimulus.cin;
    ALSU_vif.red_op_A = seq_item_stimulus.red_op_A;
    ALSU_vif.red_op_B = seq_item_stimulus.red_op_B;
    ALSU_vif.bypass_A = seq_item_stimulus.bypass_A;
    ALSU_vif.bypass_B = seq_item_stimulus.bypass_B;
    ALSU_vif.direction = seq_item_stimulus.direction;
    ALSU_vif.serial_in = seq_item_stimulus.serial_in;
    ALSU_vif.opcode = seq_item_stimulus.opcode;
    ALSU_vif.A = seq_item_stimulus.A;
    ALSU_vif.B = seq_item_stimulus.B;

    ALSU_if_golden.rst = seq_item_stimulus.rst;
    ALSU_if_golden.cin = seq_item_stimulus.cin;
    ALSU_if_golden.red_op_A = seq_item_stimulus.red_op_A;
    ALSU_if_golden.red_op_B = seq_item_stimulus.red_op_B;
    ALSU_if_golden.bypass_A = seq_item_stimulus.bypass_A;
    ALSU_if_golden.bypass_B = seq_item_stimulus.bypass_B;
    ALSU_if_golden.direction = seq_item_stimulus.direction;
    ALSU_if_golden.serial_in = seq_item_stimulus.serial_in;
    ALSU_if_golden.opcode = seq_item_stimulus.opcode;
    ALSU_if_golden.A = seq_item_stimulus.A;
    ALSU_if_golden.B = seq_item_stimulus.B;

    @(negedge ALSU_vif.clk);
    @(negedge ALSU_vif.clk);
    seq_item_port.item_done();
  end
endtask
```

-monitor

```
class ALSU_monitor extends uvm_monitor;
  `uvm_component_utils(ALSU_monitor)

  virtual ALSU_if ALSU_vif;
  virtual ALSU_if ALSU_if_golden;
  ALSU_seq_item rsp_seq_item;
  ALSU_seq_item golden_seq_item;
  uvm_analysis_port #(ALSU_seq_item) mon_ap;
  uvm_analysis_port #(ALSU_seq_item) mon_ap_golden;

  function new(string name = "ALSU_monitor", uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
    mon_ap_golden = new("mon_ap_golden", this);
  endfunction
```

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);

  forever begin
    rsp_seq_item = ALSU_seq_item::type_id::create("rsp_seq_item");
    golden_seq_item = ALSU_seq_item::type_id::create("golden_seq_item");
    @(negedge ALSU_vif.clk); //driver drives interface and the virt
    @(negedge ALSU_vif.clk);

    rsp_seq_item.rst = ALSU_vif.rst;
    rsp_seq_item.cin = ALSU_vif.cin;
    rsp_seq_item.red_op_A = ALSU_vif.red_op_A;
    rsp_seq_item.red_op_B = ALSU_vif.red_op_B;
    rsp_seq_item.bypass_A = ALSU_vif.bypass_A;
    rsp_seq_item.bypass_B = ALSU_vif.bypass_B;
    rsp_seq_item.direction = ALSU_vif.direction;
    rsp_seq_item.serial_in = ALSU_vif.serial_in;
    rsp_seq_item.opcode = ALSU_vif.opcode;
    rsp_seq_item.A = ALSU_vif.A;
    rsp_seq_item.B = ALSU_vif.B;
    rsp_seq_item.leds = ALSU_vif.leds;
    rsp_seq_item.out = ALSU_vif.out;

    golden_seq_item.leds = ALSU_if_golden.leds;
    golden_seq_item.out = ALSU_if_golden.out;

    mon_ap.write(rsp_seq_item);
    mon_ap_golden.write(golden_seq_item);
    `uvm_info("run_phase",rsp_seq_item.convert2string(), UVM_HIGH)
  end
endtask
```

-scoreboard

```
class ALSU_scoreboard extends uvm_component;
  `uvm_component_utils(ALSU_scoreboard)

  uvm_analysis_export #(ALSU_seq_item) sb_export;
  uvm_tlm_analysis_fifo #(ALSU_seq_item) sb_fifo;

  uvm_analysis_export #(ALSU_seq_item) sb_export_golden;
  uvm_tlm_analysis_fifo #(ALSU_seq_item) sb_fifo_golden;

  ALSU_seq_item seq_item_sb;
  ALSU_seq_item golden_seq_item;

  int error_count = 0;
  int correct_count = 0;

  function new(string name = "ALSU_scoreboard", uvm_component parent
    | super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_export = new("sb_export", this);
    sb_fifo = new("sb_fifo", this);
    sb_export_golden = new("sb_export_golden", this);
    sb_fifo_golden = new("sb_fifo_golden", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    sb_export.connect(sb_fifo.analysis_export);
    sb_export_golden.connect(sb_fifo_golden.analysis_export);
  endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);

  forever begin
    sb_fifo.get(seq_item_sb);
    sb_fifo_golden.get(golden_seq_item);

    if((seq_item_sb.out != golden_seq_item.out) || (seq_item_sb.leds != golden_seq_item.
      `uvm_error("run_phase", $sformatf("Comparison failed, concerning Transaction: %s |
      seq_item_sb.convert2string(), golden_seq_item.out, golden_seq_item.leds));
      error_count++;
    end else begin
      `uvm_info("run_phase", $sformatf("Transaction handled correctly: %s" , seq_item_sb
      correct_count++;
    end
  end
endtask

function void report_phase(uvm_phase phase);
  super.report_phase(phase);
  `uvm_info("report_phase", $sformatf("correct_count: %d" , correct_count), UVM_MEDIUM)
  `uvm_info("report_phase", $sformatf("error_count: %d" , error_count), UVM_MEDIUM)
endfunction
```

-Coverage collector

```

class ALSU_coverage extends uvm_component;
`uvm_component_utils(ALSU_coverage)

uvm_analysis_export #(ALSU_seq_item) cov_export;
uvm_tlm_analysis_fifo #(ALSU_seq_item) cov_fifo;
ALSU_seq_item seq_item_cov;

//CoverGroups=====
covergroup CovPort; //@(posedge clk)
  cin_cp: coverpoint seq_item_cov.cin iff(1){}
  direction_cp: coverpoint seq_item_cov.direction iff(1){}
  serial_in_cp: coverpoint seq_item_cov.serial_in iff(1){}
  red_op_A_cp: coverpoint seq_item_cov.red_op_A iff(1){}
  red_op_B_cp: coverpoint seq_item_cov.red_op_B iff(1){}

  A_cp: coverpoint seq_item_cov.A iff(1)
  {
    bins A_data_0 = {ZERO};
    bins A_data_max = {MAXPOS};
    bins A_data_min = {MAXNEG};
    bins A_data_default = default;
    bins A_data_walkingones[] = {3'b001,3'b010,3'b100} iff(seq_item_cov.red_op_A);
  }

  B_cp: coverpoint seq_item_cov.B iff(1)
  {
    bins B_data_0 = {ZERO};
    bins B_data_max = {MAXPOS};
    bins B_data_min = {MAXNEG};
    bins B_data_default = default;
    bins B_data_walkingones[] = {3'b001,3'b010,3'b100} iff(!seq_item_cov.red_op_A && seq_item_cov.red_op_B);
  }

  ALU_cp: coverpoint seq_item_cov.opcode iff(1)
  {
    bins Bins_shift[] = {SHIFT,ROTATE};
    bins Bins_arith[] = {ADD, MULT};
    bins Bins_bitwise[] = {OR, XOR};
    illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
    bins Bins_trans = (0 => 1 => 2 => 3 => 4 => 5);
  }
//=====Cross Coverage=====

//1.
cross ALU_cp,A_cp{
  bins Arith_corners_A_1 = binsof(ALU_cp.Bins_arith) &&
  | binsof(A_cp.A_data_0);
  bins Arith_corners_A_2 = binsof(ALU_cp.Bins_arith) &&
  | binsof(A_cp.A_data_max);
  bins Arith_corners_A_3 = binsof(ALU_cp.Bins_arith) &&
  | binsof(A_cp.A_data_min);
  option.cross_auto_bin_max = 0;
}

cross ALU_cp,B_cp{
  bins Arith_corners_B_1 = binsof(ALU_cp.Bins_arith) &&
  | binsof(B_cp.B_data_0);
  bins Arith_corners_B_2 = binsof(ALU_cp.Bins_arith) &&
  | binsof(B_cp.B_data_max);
  bins Arith_corners_B_3 = binsof(ALU_cp.Bins_arith) &&
  | binsof(B_cp.B_data_min);
  option.cross_auto_bin_max = 0;
}

//2.
cross ALU_cp,cin_cp{
  bins Add_cin = binsof(ALU_cp.Bins_arith[0]) &&
  | binsof(cin_cp);
  option.cross_auto_bin_max = 0;
}

//3.
cross ALU_cp,direction_cp{
  bins Shifts_direction = binsof(ALU_cp.Bins_shift) &&
  | binsof(direction_cp);
  option.cross_auto_bin_max = 0;
}

```



```

// =====
// ADD
// =====
property p_add_full_adder_on;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h2) &&
  (FULL_ADDER == "ON")
  |-> (ALSU_if.out ===
      ($signed($past(ALSU_if.A,2)) +
       $signed($past(ALSU_if.B,2)) +
       {5'b0, $past(ALSU_if.cin,2)}));
endproperty

property p_add_full_adder_off;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h2) &&
  (FULL_ADDER != "ON")
  |-> (ALSU_if.out ===
      ($signed({3{$past(ALSU_if.A[2],2)}}, $past(ALSU_if.A,2)) +
       $signed({3{$past(ALSU_if.B[2],2)}}, $past(ALSU_if.B,2))));
endproperty

// =====
// MULT
// =====
property p_mult;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h3)
  |-> (ALSU_if.out === ($past(ALSU_if.A,2) * $past(ALSU_if.B,2)));
endproperty

```

```

// =====
// SHIFT
// =====
property p_shift_left;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h4) &&
  $past(ALSU_if.direction,2)
  |-> (ALSU_if.out === {$past(ALSU_if.out[4:0],1), $past(ALSU_if.serial_in,2)});
endproperty

property p_shift_right;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h4) &&
  !$past(ALSU_if.direction,2)
  |-> (ALSU_if.out === {$past(ALSU_if.serial_in,2), $past(ALSU_if.out[5:1],1)});
endproperty

// =====
// ROTATE
// =====
property p_rotate_left;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h5) &&
  $past(ALSU_if.direction,2)
  |-> (ALSU_if.out === {$past(ALSU_if.out[4:0],1), $past(ALSU_if.out[5],1)});
endproperty

property p_rotate_right;
  @(posedge ALSU_if.clk)
  disable iff (ALSU_if.rst)
  !$past(ALSU_if.rst,1) && !$past(ALSU_if.rst,2) &&
  !$past(ALSU_if.bypass_A,2) && !$past(ALSU_if.bypass_B,2) &&
  !$past(ALSU_if.red_op_A,2) && !$past(ALSU_if.red_op_B,2) &&
  ($past(ALSU_if.opcode,2) == 3'h5) &&
  !$past(ALSU_if.direction,2)
  |-> (ALSU_if.out === {$past(ALSU_if.out[0],1), $past(ALSU_if.out[5:1],1)});
endproperty

```

```

assert property (p_reset_out_zero) else $error("RESET out failed");
assert property (p_reset_leds_zero) else $error("RESET leds failed");

assert property (p_bypass_A_and_B_priority_A) else $error("Bypass A&B priority A failed");
assert property (p_bypass_A_and_B_priority_B) else $error("Bypass A&B priority B failed");
assert property (p_bypass_A_only) else $error("Bypass A failed");
assert property (p_bypass_B_only) else $error("Bypass B failed");

assert property (p_invalid_out_zero) else $error("Invalid out failed");
assert property (p_invalid_leds_toggle) else $error("Invalid leds toggle failed");
assert property (p_valid_leds_zero) else $error("Valid leds zero failed");

assert property (p_or_redA_redB_priority_A) else $error("OR reduction A priority failed");
assert property (p_or_redA_redB_priority_B) else $error("OR reduction B priority failed");
assert property (p_or_redA_only) else $error("OR red_op_A failed");
assert property (p_or_redB_only) else $error("OR red_op_B failed");
assert property (p_or_normal) else $error("OR normal failed");

assert property (p_xor_redA_redB_priority_A) else $error("XOR reduction A priority failed");
assert property (p_xor_redA_redB_priority_B) else $error("XOR reduction B priority failed");
assert property (p_xor_redA_only) else $error("XOR red_op_A failed");
assert property (p_xor_redB_only) else $error("XOR red_op_B failed");
assert property (p_xor_normal) else $error("XOR normal failed");

assert property (p_add_full_adder_on) else $error("ADD full adder ON failed");
assert property (p_add_full_adder_off) else $error("ADD full adder OFF failed");
assert property (p_mult) else $error("MULT failed");

assert property (p_shift_left) else $error("SHIFT left failed");
assert property (p_shift_right) else $error("SHIFT right failed");
assert property (p_rotate_left) else $error("ROTATE left failed");
assert property (p_rotate_right) else $error("ROTATE right failed");

```

-Agent

```
class ALSU_agent extends uvm_agent;
  // Example 1 (Done for you)
  // Do the essentials (factory register & Constructor)
  `uvm_component_utils(ALSU_agent) //2.

  ALSU_config ALSU_cfg;

  ALSU_driver driver;
  ALSU_sqr sqr;
  ALSU_monitor monitor;

  uvm_analysis_port #(ALSU_seq_item) agt_ap;
  uvm_analysis_port #(ALSU_seq_item) agt_ap_golden;

  //3.
  function new(string name = "ALSU_agent", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  // Example 2
  // Build the driver in the build phase

  //4.
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    if(!uvm_config_db #(ALSU_config)::get(this, "", "CFG", ALSU_cfg))
      `uvm_fatal("build_phase", "Can't get Interface from database!!!");

    driver = ALSU_driver::type_id::create("driver", this);
    sqr = ALSU_sqr::type_id::create("sqr", this);
    monitor = ALSU_monitor::type_id::create("monitor", this);
    agt_ap = new("agt_ap", this);
    agt_ap_golden = new("agt_ap_golden", this);
  endfunction

function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);

  driver.ALSU_vif = ALSU_cfg.ALSU_vif;
  driver.ALSU_if_golden = ALSU_cfg.ALSU_if_golden;
  monitor.ALSU_vif = ALSU_cfg.ALSU_vif;
  monitor.ALSU_if_golden = ALSU_cfg.ALSU_if_golden;

  monitor.mon_ap.connect(agt_ap);
  monitor.mon_ap_golden.connect(agt_ap_golden);
  driver.seq_item_port.connect(sqr.seq_item_export);
endfunction
```

-Environment

```
class ALSU_env extends uvm_env;
  // Example 1 (Done for you)
  // Do the essentials (factory register & Constructor)
  `uvm_component_utils(ALSU_env) //2.
```

```
  ALSU_agent agt;
  ALSU_scoreboard sb;
  ALSU_coverage cov;
```

```
//4.
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  agt = ALSU_agent::type_id::create("agt", this);
  sb = ALSU_scoreboard::type_id::create("sb", this);
  cov = ALSU_coverage::type_id::create("cov", this);
endfunction

function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);

  agt.agt_ap.connect(cov.cov_export);
  agt.agt_ap.connect(sb.sb_export);
  agt.agt_ap_golden.connect(sb.sb_export_golden);

endfunction
```

-cfg Object

```
class ALSU_config extends uvm_object;
  `uvm_object_utils(ALSU_config)

  virtual ALSU_if ALSU_vif;
  virtual ALSU_if ALSU_if_golden;

  function new(string name = "ALSU_config");
    super.new(name);
  endfunction
```

-enums

```
package enum_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"

typedef enum reg [2:0] {OR, XOR, ADD, MULT,
  SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;

localparam MAXPOS = 3, ZERO = 0, MAXNEG = -4;

endpackage
```

-Test

```
class ALSU_test extends uvm_test;
  `uvm_component_utils(ALSU_test) //2.
  // Example 1
  // Do the essentials (factory register & Constructor)
  // Build the environment in the build phase
  // Run in the test in the run phase, raise objection, add #100 delay then display a message using
  ALSU_env env;
  ALSU_config ALSU_cfg;

  ALSU_reset_seq reset_seq;
  ALSU_Constrained_main_seq seq_1;
  ALSU_Heavy_opcode_verification_seq seq_2;

  //3.
  function new(string name = "ALSU_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  //4.
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = ALSU_env::type_id::create("env", this);
    ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg");

    reset_seq = ALSU_reset_seq::type_id::create("reset_seq", this);
    seq_1 = ALSU_Constrained_main_seq::type_id::create("seq_1", this);
    seq_2 = ALSU_Heavy_opcode_verification_seq::type_id::create("seq_2", this);

    if(!uvm_config_db #(virtual ALSU_if)::get(this, "", "ALSU_IF", ALSU_cfg.ALSU_vif))
      `uvm_fatal("build_phase", "Can't get Interface from database!!!");
    if(!uvm_config_db #(virtual ALSU_if)::get(this, "", "ALSU_IF_Golden", ALSU_cfg.ALSU_if_golden))
      `uvm_fatal("build_phase", "Can't get Interface from database!!!");

    uvm_config_db #(ALSU_config)::set(this, "*", "CFG", ALSU_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);

    `uvm_info("run_phase", "Reset asserted" , UVM_LOW)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Reset deasserted" , UVM_LOW)

    `uvm_info("run_phase", "Full constrained Randomization generation started" , UVM_LOW)
    seq_1.start(env.agt.sqr);
    `uvm_info("run_phase", "Full constrained Randomization generation done" , UVM_LOW)

    `uvm_info("run_phase", "Pure randomization for each valid opcode generation started" , UVM_LOW)
    seq_2.start(env.agt.sqr);
    `uvm_info("run_phase", "Pure randomization for each valid opcode generation done" , UVM_LOW)

    phase.drop_objection(this);
  endtask
```

Results

-Do File

```
#vlib work
vlog -sv -f ../../VS_code/Session6/Assignment6/src_files.list +cover -covercells
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
#add wave /top/ALU_if/*
coverage save work.ucdb -onexit
do wave.do
run -all

vcover report work.ucdb -details -all -output code_cvr.txt
```

-Code Coverage

```
=====
=== File: ../../VS_code/Session6/Assignment6/ALU.v
=====
Statement Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Stmts                51         48         3      94.1
```

Justified dead code

```
***0***      3'h6: out <= 0;
***0***      3'h7: out <= 0;
***0***      default: out <= 0;
```

```
Branch Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Branches              34         31         3      91.1
```

```
Toggle Coverage:
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Toggle Bins           118        118         0     100.0
```

-CoverGroups

INST \ALSU_coverage_pkg::ALSU_c...	100.0%	100	100.0%		✓
+ CVP cin_cp	100.0%	100	100.0%		✓
+ CVP direction_cp	100.0%	100	100.0%		✓
+ CVP serial_in_cp	100.0%	100	100.0%		✓
+ CVP red_op_A_cp	100.0%	100	100.0%		✓
+ CVP red_op_B_cp	100.0%	100	100.0%		✓
+ CVP A_cp	100.0%	100	100.0%		✓
+ CVP B_cp	100.0%	100	100.0%		✓
+ CVP ALU_cp	100.0%	100	100.0%		✓
+ CROSS #cross__0#	100.0%	100	100.0%		✓
+ CROSS #cross__1#	100.0%	100	100.0%		✓
+ CROSS #cross__2#	100.0%	100	100.0%		✓
+ CROSS #cross__3#	100.0%	100	100.0%		✓
+ CROSS #cross__4#	100.0%	100	100.0%		✓
+ CROSS #cross__5#	100.0%	100	100.0%		✓
+ CROSS #cross__6#	100.0%	100	100.0%		✓
+ CROSS #cross__7#	100.0%	100	100.0%		✓

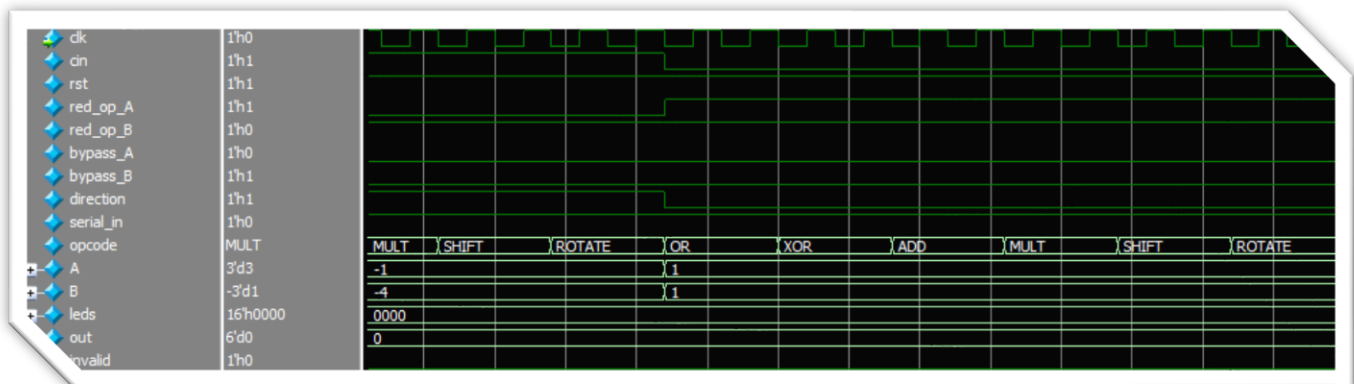
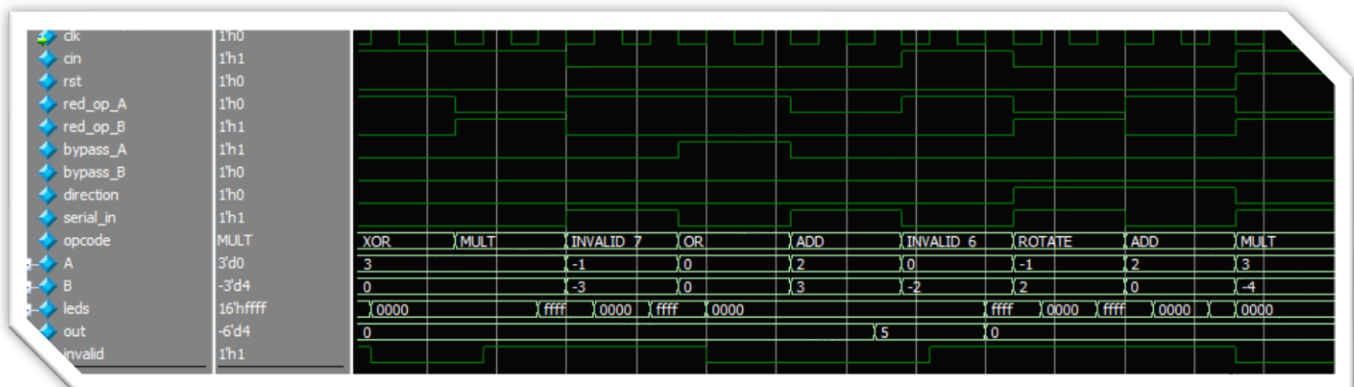
-Assertions

```

assert (randomize(...)) ✓
assert (randomize(...)) ✓
assert (randomize(...)) ✓
assert( @(posedge ALSU_if.dk) (ALSU_if.rst)->ALSU_if.out==0) ✓
assert( @(posedge ALSU_if.dk) (ALSU_if.rst)->ALSU_if.leds==0) ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(A... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) ((((((~$past(A... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pas... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓
assert( @(posedge ALSU_if.dk) disable iff (ALSU_if.rst) (((((((~$pa... ✓

```

-Waveforms



Drive link with code:

https://drive.google.com/drive/folders/1FofcrOVPlZNVkbfJge_Pep1GBzMVHcGX?usp=sharing